Online Float-sat

Inertial Measurement Unit „I2C“

M.Sc. Eng. Atheel Redah

Inertial Measurement Unit (IMU):

● Inertial Measurement Unit (IMU), is an electronic device that measures and reports on an object's velocity, orientation, and gravitational forces, using a combination of accelerometers, gyroscopes, and magnetometers.

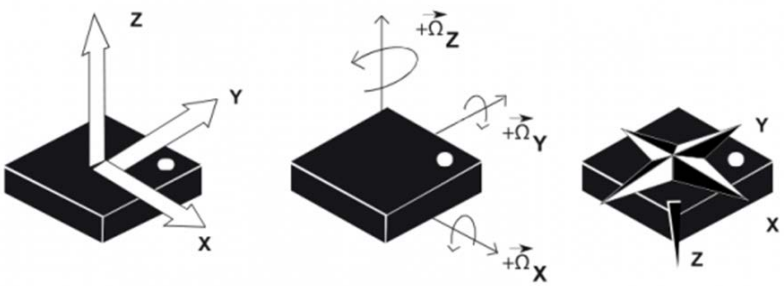
● The LSM9DS0 is an inertial measurement unit (IMU) that packs an 3-axis Gyroscope with 3-axis Accelerometer and 3-axis Magnetometer:

* SPI / I2C serial interfaces.

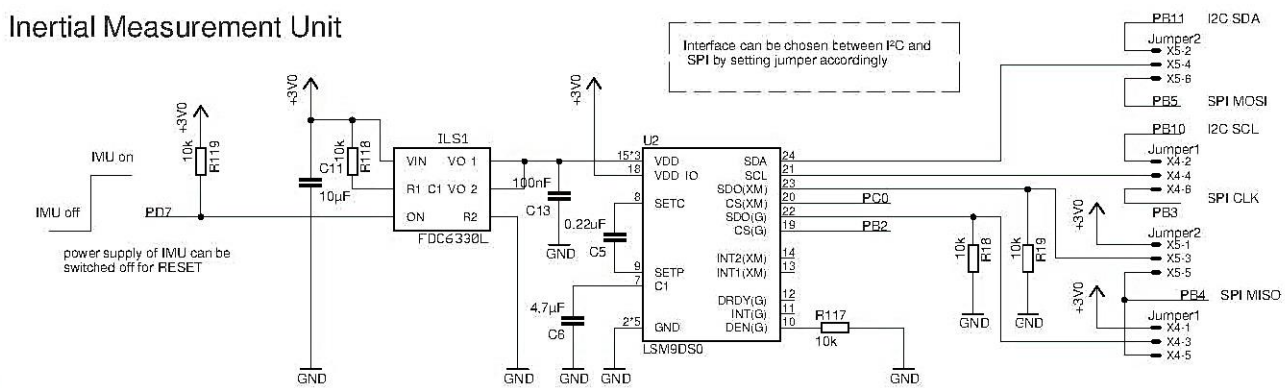
* 16-bit data output per axis.

* Sensitivity range (configurable):
 - Gyro: ±245, ±500, or ±2000 dps angular rate full scale.
 - Accelerometer: ±2, ±4, ±6, ±8 or ±16 g linear acceleration full scale.
 - Magnetometer: ±2, ±4, ±8, or ±12 gauss magnetic full scale.

* Embedded temperature sensor.



● Orientation Angles:
- Three attitude angles are referenced to the local horizontal plane which is perpendicular to earth's gravity:
* Heading is defined as the angle between the X axis and the magnetic north on the horizontal plane.
* Pitch is defined as the angle between the X axis and the horizontal plane.
* Roll is defined as the angle between the Y axis and the horizontal plane.



Inertial Measurement Unit (IMU):

● Accelerometer Sensor:

● Operation:

- Accelerometers are sensors or transducers that measure linear acceleration.
- Accelerometers generally measure linear acceleration forces applied to a body by being mounted directly onto a surface of the accelerated body and also measure the tilt angles of the body: pitch, and roll.
- An accelerometer measures force exerted by a body as a result of a change in the velocity of the body. A moving body possesses an inertia which tends to resist change in velocity. It is this resistance to change in velocity that is the source of the force exerted by the moving body. This force is directly proportional to the acceleration component in the direction of movement when the moving body is accelerated.

● Calibration:

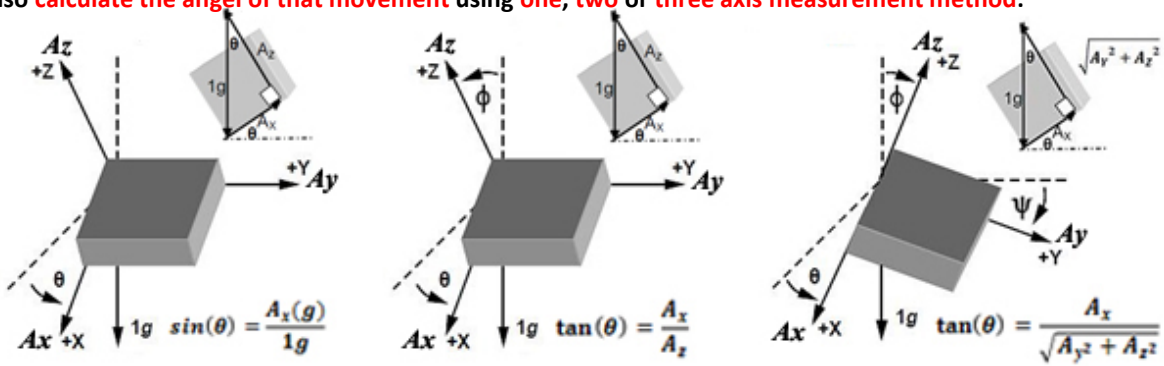
A simple calibration process for a 3-Axis Accelerometer sensor can be performed by taking N samples for each axis at three positions in standstill condition: P1->[0g X, 0g Y, 1g Z], P2->[0g X, 1g Y, 0g Z] and P3->[1g X, 0g Y, 0g Z] and then average them to get two offset values correspond to 0g measurement for each axis. The two offset values correspond to 0g measurement of each axis can then be averaged to give the final offset values for each axis.

● Measurement:

There are two types of measurements can be preform using an accelerometer sensor:

* Dynamic Acceleration: In this type of measurement, the sensor measures linear acceleration of a moving body. As The sensor move along an axis, the value corresponds to that axis changes. By knowing the initial offset value of the sensor (standstill value) and by measuring the current value of the sensor continuously, we can know if the sensor is on the move and also we can calculate the acceleration of that movement in g using the sensitivity value provided by the datasheet.

* Static Acceleration: In this type of measurement, the sensor measure the magnitude of earth's gravity field mounted on it, which is very useful to get orientation information of an object. As the sensor change its orientation along an axis, the sensor value corresponds to that axis changes. By knowing the initial offset value of the sensor (standstill value) and by measuring the current value of the sensor continuously, we can know if the sensor has change its orientation along that axis and we can also calculate the angel of that movement using one, two or three axis measurement method:



Inertial Measurement Unit (IMU):

● Magnetometer Sensor:

● Operation:

- Magnetometers are sensors that measure magnetic field *B*, which is also known as the magnetic flux density or the magnetic induction.
- Magnetometers are useful in measuring Earth's magnetic field components which can be used to calculate the magnetic north of the earth (magnetic compass).

● Calibration:

A simple Hard-Iron calibration and normalization process for a 3-Axis Magnetometer sensor can be performed by measuring maximum and minimum values for each axis while rotating the sensor around the axis. These max and min values can be used to shift and scale the axis values as follows:

$$Mx_s = \frac{Mx - Mx_{min}}{Mx_{max} - Mx_{min}} * 2 - 1 \qquad My_s = \frac{My - My_{min}}{My_{max} - My_{min}} * 2 - 1 \qquad Mz_s = \frac{Mz - Mz_{min}}{Mz_{max} - Mz_{min}} * 2 - 1$$

● Measurement:

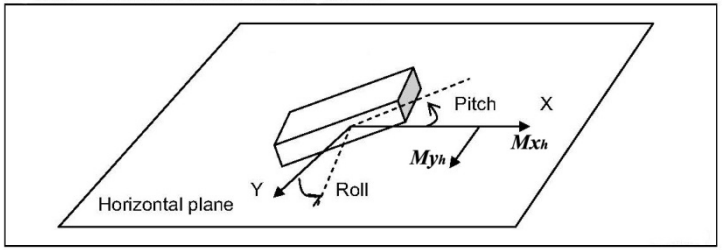
There are two types of measurements can be preform using a magnetometer sensor:

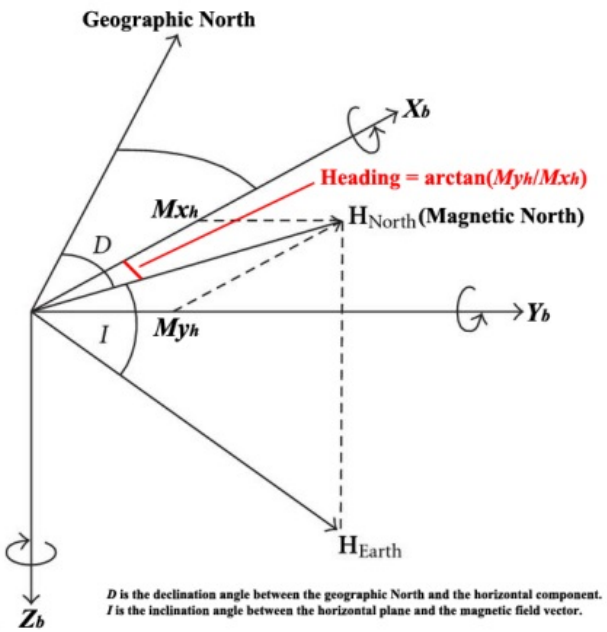
* Magnetic Flux Density: the sensor can measure the magnetic flux density along each axis in Gauss using the sensitivity value provided by the datasheet.

* Compass Angle: the orientation angle of the magnetic north for an object can be calculated using the horizontal components of the magnetometer.

* Tilt Compensation: if the sensor is tilted, the magnetic sensor measurements need to be compensated to obtain the horizontal components Mx_h and My_h using following equations:

$$Mx_h = Mx_s.\cos(pitch) + Mz_s.\sin(pitch)$$
$$My_h = Mx_s.\sin(roll).\sin(pitch) + My_s.\cos(roll) - Mz_s.\sin(roll).\cos(pitch)$$





Inertial Measurement Unit (IMU):

● Gyroscope Sensor:

● Operation:

- Gyroscopes are sensors or transducers that measure angular velocity.
- Gyroscopes are useful in detecting rotation motion in objects and also in measuring orientation angles of the object: pitch, roll and yaw(heading).
- Microelectromechanical System (MEMS) Gyroscop uses the phenomenon of Coriolis force, which is generated when a rotational angular velocity is applied to the oscillating body where the vibrating object tends to continue vibrating in the same plane as its support rotates.

● Calibration:

A simple calibration process for a 3-Axis Gyroscope sensor can be performed by taking N samples for each axis in standstill condition and then average them to get the offset value correspond to 0 deg/sec measurement for each axis.

● Measurement:

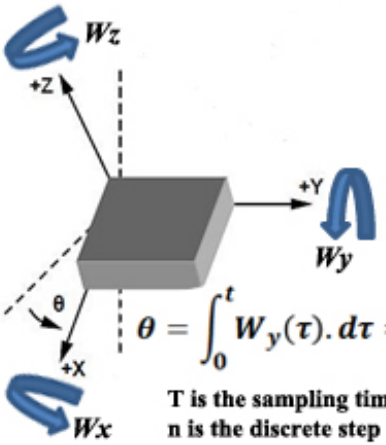
There are two types of measurements can be preform using a gyroscope sensor:

* Angular velocity: the sensor measures the angular velocity of a rotating body. As The sensor rotate around an axis, the value corresponds to that axis changes. By knowing the initial offset value of the sensor (standstill value) and by measuring the current value of the sensor continuously, we can know if the sensor is rotating and also we can calculate the angular velocity of that movement in deg/sec using the sensitivity value provided by the datasheet.

* Orientation Angles:

-> The orientation angle of an object expressed in the sensor body frame can be calculated by integrating the angular velocity measured by the gyroscope over a period of time.

-> In order to get the orientation angles expressed in Euler convention, the following equations can be used:



$$\theta = \int_0^t W_y(\tau).d\tau = T.\sum_{k=0}^{n} W_y(k)$$

T is the sampling time period in seconds
n is the discrete step at time t

$$\begin{bmatrix} \frac{\delta pitch}{\delta t} \\ \frac{\delta roll}{\delta t} \\ \frac{\delta heading}{\delta t} \end{bmatrix} = \frac{1}{\cos(pitch)} . \begin{bmatrix} 0 & \cos(roll)\cos(pitch) & -\sin(roll)\cos(pitch) \\ \cos(pitch) & \sin(roll)\sin(pitch) & \cos(roll)\sin(pitch) \\ 0 & \sin(roll) & \cos(roll) \end{bmatrix} . \begin{bmatrix} w_x \\ w_y \\ w_z \end{bmatrix}$$

$$\begin{bmatrix} pitch(t) \\ roll(t) \\ heading(t) \end{bmatrix} = \begin{bmatrix} pitch(t-1) \\ roll(t-1) \\ heading(t-1) \end{bmatrix} + \begin{bmatrix} \frac{\delta pitch}{\delta t} \\ \frac{\delta roll}{\delta t} \\ \frac{\delta heading}{\delta t} \end{bmatrix} . T$$

Inter-Integrated Circuit (I2C) Interface:

● **I2C (inter-integrated circuit)** bus Interface serves as an interface between the microcontroller and the serial I2C bus. It controls all I2C bus-specific sequencing, protocol, arbitration and timing. The interface is connected to the I2C bus by a **data pin (SDA)** and by a **clock pin (SCL)**.

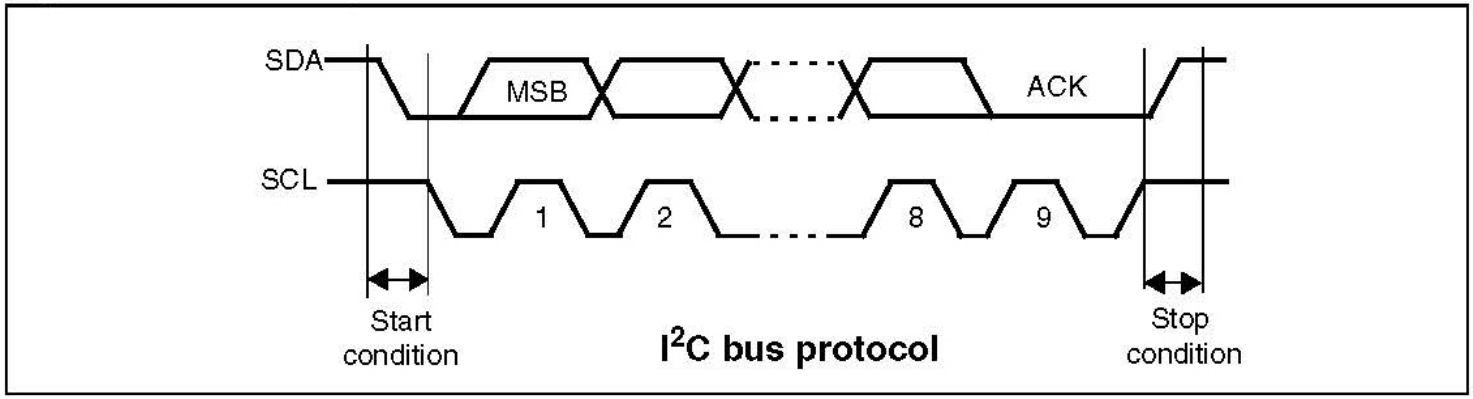
● STM32F4 I2C Main Features :

- Up to **three I²C bus** interfaces with parallel-bus/I2C protocol converter.
- Can operate in **multimaster** (the same interface can act as Master or Slave) and **slave** modes.
- **I2C Master features:** **Clock generation**, and **Start and Stop generation**.
- **I2C Slave features:** **Programmable I2C Address detection**, **Dual Addressing Capability to acknowledge 2 slave addresses**, and **Stop bit detection**.
- Generation and detection of **7-bit/10-bit addressing** and **General Call**.
- Supports different communication speeds: **Standard Speed (up to 100 kHz)**, and **Fast Speed (up to 400 kHz)**.
- Optional **clock stretching**.
- **1-byte buffer** with **DMA** capability.
- A hardware **CRC generation/verification** is embedded.
- **SMBus 2.0** (System Management Bus) / **PMBus** (Power Management Bus) Compatibility.
- **Status flags:** **Transmitter/Receiver mode flag**, and **End-of-Byte transmission flag**, and **I2C busy flag**.
- **Error flags:** **Arbitration lost condition for master mode**, **Acknowledgement failure after address/ data transmission** , **Detection of misplaced start or stop condition**, and **Overrun/Underrun if clock stretching is disabled**.
- **Interrupt vectors:** **Interrupt for successful address/ data communication**, **Interrupt for error condition**.

Inter-Integrated Circuit (I2C) Interface:

- **Communication flow:**

- In **Master mode**, the I2C interface **initiates a data transfer** and **generates the clock signal**.
- A serial data transfer always **begins** with a **start condition** and **ends** with a **stop condition**. Both start and stop conditions are generated in master mode by software.
- Data and addresses are transferred as **8-bit bytes**, **MSB first**.
- The **first byte(s)** following the start condition contain **the address (one in 7-bit mode, two in 10-bit mode)**. The address is always transmitted in Master mode.
- A **9th clock pulse** follows the 8 clock cycles of a byte transfer, during which the **receiver** must send an **acknowledge bit** to the transmitter.
- In **Slave mode**, the interface is capable of **recognizing** its **own addresses (7 or 10-bit)**, and the **General Call address**. The General Call address detection may be enabled or disabled by software.



I²C bus protocol

RODOS HAL (I2C):

```
class HW_HAL_I2C; /* hal_i2c.h */
class HAL_I2C : public GenericIOInterface {
HW_HAL_I2C *context;
public:
HAL_I2C(I2C_IDX idx);
 /** Initialize I2C interface default: master, 7Bit address * @param speed I2C bus speed * @retval int32_t zero on success **/
int32_t init(uint32_t speed = 100000);
void reset();
bool isWriteFinished();
bool isReadFinished();

 /** Transmit data: @param addr I2C address of chip, txBuf pointer to transmit buffer, txBufSize size of transmit buffer
  * @retval int32_t number of transmitted bytes **/
int32_t write(const uint8_t addr, const uint8_t *txBuf, uint32_t txBufSize);

 /** Receive data: @param addr I2C address of chip, rxBuf pointer to receive buffer, rxBufSize size of receive buffer
  * @retval int32_t number of received bytes **/
int32_t read(const uint8_t addr, uint8_t *rxBuf, uint32_t rxBufSize);

 /** Send and receive data without a STOP condition between: @param addr I2C address of chip, txBuf pointer to transmit buffer,
txBufSize size of transmit buffer, rxBuf pointer to receive buffer, rxBufSize size of receive buffer
  * @retval int32_t number of received bytes **/
int32_t writeRead(const uint8_t addr, const uint8_t *txBuf, uint32_t txBufSize, uint8_t *rxBuf, uint32_t rxBufSize);
};
```

Example:

```
#define LSM9DS0_G                0x6B                /* Gyro 7-bit I2C address*/
uint8_t LSM9DS0_WHO_AM_I_G[1] = {0x0F};              /* Gyro WHO_AM_I register address*/
uint8_t LSM9DS0_CTRL_REG1_G[2] = {0x20,0b11111111}; /* { Gyro CTRL_REG1 register address, Vule needs to be written*
uint8_t DATA[1];
HAL_GPIO CS_G(GPIO_018);       /* declare HAL_GPIO for GPIO_018 = PB2 (IMU Chip Select pin for the Gyro) */
HAL_GPIO IMU_EN(GPIO_055);     /* declare HAL_GPIO for GPIO_055 = PD7 (IMU Power Enable pin) */
HAL_I2C IMU(I2C_IDX2);         /* declare HAL_I2C for I2C_IDX2 = I2C2 (IMU), please refer to hal_i2c.h for correct I2C mapping */

CS_G.init(true, 1, 1);         /* initialization of the HAL object should be called one time only in the project*/
IMU_EN.init(true, 1, 1);       /* initialization of the HAL object should be called one time only in the project*/
IMU.init(400000);              /* initialization of the HAL object should be called one time only in the project*/

LSM9DS0_WHO_AM_I_G[0] = LSM9DS0_WHO_AM_I_G[0] | 0x00; /*If single byte, bit 0 of the subAddress should be 0 (0x00), otherwise 1 (0x80)*/

IMU.write(LSM9DS0_G, LSM9DS0_CTRL_REG1_G, 2);  /* Write to the gyroscope CTRL_REG1 register (0x20) */
IMU.writeRead(LSM9DS0_G, LSM9DS0_WHO_AM_I_G, 1, DATA, 1);  /* Read from the gyroscope WHO_AM_I register (0xD4) */
PRINTF("LSM9DS0_WHO_AM_I_G = %x \r\n", DATA[0]);
```

• Tasks:

3.1 IMU:

Implement a project with following requirements:

• Implement an initialization function to initialize the sensors to work in specific operation modes by writing to specific registers in the sensors through the I2C interface as following (please refer to the datasheet of the sensors for the corresponding values):

Register	Adresse	Mode
CTRL_REG1_G	0x20	ODR=190Hz, Cutoff=12.5Hz, Normal Mode, Gyro Axis Enabled
CTRL_REG4_G	0x23	BDU Enabled, Data LSb @ Lower Address, Full-scale = 2000 dps, Self-test Disabled
CTRL_REG1_XM	0x20	Acceleration ODR=200Hz, BDU Enabled, Acceleration Axis Enabled
CTRL_REG2_XM	0x21	Acceleration Anti-alias Filter Bandwidth=50Hz, Acceleration Full-scale = +-2g, Normal Mode
CTRL_REG5_XM	0x24	Temperature Sensor Enable, Magnetic Resolution Low, Magnetic Data Rate = 100 Hz
CTRL_REG6_XM	0x25	Magnetic Full-scale = +-2 gauss
CTRL_REG7_XM	0x26	Normal Mode, Internal Filter Bypassed, Continuous Conversion Mode

• Implement a reset function to reset the sensors and the I2C interface in case of an error is detected during reading or writing operations.

• Implement three different calibration modes for the three sensors of the IMU that can be activated with telecommands when needed. At the end of each calibration mode, the calculated offset values should be send through UART3.

• Tasks:

• Create 3 threads in the project, each of them should toggle different LED when it is running and perform the following:

>> The first thread is the Signal Processing thread that needs to run with following requirements:

• An initialization of the sensors mounted in the extension board should be performed.
• A data gathering process to get the updated values of the three axis of the sensors, should be performed continuously at 10 millisecond time interval using the starting data register addresses of the sensors.
• A signal processing for the gathered information to get the angular velocity in deg/sec, angular position in deg, the linear acceleration in g, and the magnetic flux density in Gauss for all axis and the Temperature in °C using the sensors sensitivity values. In addition, the orientation angles of the Gyroscope expressed in Euler convention and the orientation angles of the accelerometer & magnetometer should be calculated.

>>The second thread is the Telemetry thread where the angular velocity in deg/sec, angular position in deg, the linear acceleration in g, the magnetic flux density in Gauss for all axis and the Temperature in °C plus the orientation angles expressed in Euler convention of the Gyroscope and the orientation angles of the accelerometer & magnetometer all in the range between 0-to-360 degree are the telemetry data need to be sent through UART3 when needed.

>>The third thread is the Telecommand thread where the signal processing thread time interval and the telemetry thread time interval should be able to update with telecommands.

• Tasks:

Tips:

- The **7-bit addresses** for the sensors are: **LSM9DS0_G=0x6B** and , **LSM9DS0_XM=0x1D**.
- The **IMU Power Enable** pin (**PD7**), the **Gyro Chip Select** pin (**PB2**) and the **XM Chip Select** pin (**PC0**) must set to **High** in order to be able to communicate with the sensors through the I2C bus.
- The **data** of the **IMU** is updated inside **6 registers** in each sensor. In order to retrieve these information, **6 bytes** need to be **read** which represents the **three axis** information as two's complement data in **16-bit format, left justified [OUT_X_L, OUT_X_H, OUT_Y_L, OUT_Y_H, OUT_Z_L, OUT_Z_H]**.
- The **data** of the **temperature sensor** is updated inside **2 registers**. In order to retrieve these information, **2 bytes** need to be **read** which represents the **temperature** information as two's complement data in **12-bit format, right justified [OUT_TEMP_L_XM, OUT_TEMP_H_XM]**.
- The **starting data register addresses** for the sensors are: **OUT_X_L_G=0x28**, **OUT_X_L_A=0x28**, **OUT_X_L_M=0x08**, and **OUT_TEMP_L_XM=0x05**.
- In order to **write/read multiple bytes** to/from the sensor (I2C Burst Mode), the **MSB (Most Significant Bit)** of the **starting data register address** should be set to **1** to indicate **auto address increment** for the sensor.
- In order to get **orientation angles** expressed in **Euler convention** using gyroscope, the **angular velocity in rad/sec** for the three axis need to be calculated using the **sensor sensitivity value** from the **datasheet** that correspond to the used operation scale.
- The **temperature sensor output change vs. temperature** is **8 LSB/°C**.